

Raiden: A Terminal Game Engine Demonstrating Core Operating System Principles

Prince Sahoo
Roll No. 230060

Dibyajyoti Behera
Roll No. 230052

Ritik Raj
Roll No. 230106

Abstract—This paper presents Raiden, a terminal-based arcade game engine written in C that demonstrates core operating system principles through its constraint-driven design. The engine avoids standard library functions for file I/O, string manipulation, and runtime dynamic memory, instead relying on custom implementations and POSIX system calls. This paper maps the engine’s architecture to five OS concepts: memory management via a pool allocator, process management via a state machine, I/O management via raw terminal mode and double-buffered rendering, file system operations via POSIX descriptors, and error handling via bounds checking and graceful halt mechanisms.

Index Terms—Operating Systems, Memory Management, Terminal I/O, POSIX, Game Engine, Finite State Machine

I. INTRODUCTION

Raiden is a vertical-scrolling shoot-’em-up that runs entirely within a 120×38 character terminal window. Its design enforces strict constraints: no `<string.h>`, no `<stdio.h>` for file I/O, no runtime `malloc()`, and no external libraries such as ncurses or SDL. These constraints force the implementation of custom subsystems that closely mirror OS-level abstractions.

The motivation behind this project is twofold. First, by building every subsystem from scratch, the developers gain a concrete understanding of how operating systems manage memory, handle device I/O, schedule processes, and maintain persistent storage. Second, the constraint-driven approach produces an engine with fully predictable resource usage—no hidden allocations, no buffered I/O surprises, and no dependency on third-party library behavior.

The codebase is organized into two layers. A `libs/` directory provides five foundational libraries: a memory pool allocator, integer math utilities, string operations, a double-buffered screen renderer, and a raw keyboard input handler. A `src/` directory contains the game logic, entity systems, UI screens, and file persistence modules. All tunable constants reside in a single `config.h` header. The project compiles with `gcc` under C11 (`-Wall -Wextra`) and produces a single binary.

A. System Overview

When the binary is executed, five subsystems initialize in a strict order: (1) the memory pool is allocated, (2) the keyboard is switched to raw mode, (3) the screen double-buffer is created, (4) the user profile system is initialized, and (5) the previous session is restored. This order is critical—the

screen allocator depends on the memory pool, and user loading depends on the file system. After initialization, control enters a finite state machine that dispatches to handler functions for each screen (title, menu, gameplay, game over, leaderboard). The game loop within the Playing state runs at a fixed 30 FPS, processing input, updating entities, resolving collisions, and rendering each frame in a deterministic pipeline.

II. METHODOLOGY

A. Memory Management

The engine replaces the C heap with a custom **pool allocator**. At boot, a single `malloc()` call claims a contiguous 64 KB block—the only standard heap allocation in the program. All subsequent allocations are carved from this pool using a custom `my_alloc()` function.

Each allocation is prefixed by an 8-byte header recording its size and free/in-use status. When memory is requested, the allocator first performs a **first-fit search** over previously freed blocks. If no suitable block is found, it bump-allocates from the unused tail of the pool. Deallocation via `my_dealloc()` simply marks the header as free in $O(1)$ time, making the block available for immediate reuse.

This design eliminates heap fragmentation because the game allocates fixed-size objects (player struct, screen buffers) that are reused at exactly the same size across replays. The two screen buffers (4,560 bytes each) are allocated once at boot and persist until exit. The player struct is allocated per game round and reused via first-fit on replay.

B. Process Management

Raiden is a single-threaded, single-process application. It manages its internal “processes” through two mechanisms:

State Machine Scheduler. The outer loop acts as a cooperative scheduler. A `GameState` enum defines seven states: Title, Username, Menu, Playing, Game Over, Leaderboard, and Quit. Each state has a handler function that blocks until resolved, then returns the next state. The main loop simply dispatches to the current handler—analogueous to cooperative multitasking where each task voluntarily yields.

Fixed-Timestep Game Loop. Inside the Playing state, a deterministic six-phase pipeline executes every frame: (1) capture input, (2) update entities, (3) resolve player-to-enemy hits, (4) resolve enemy-to-player hits, (5) update score, and (6) render. Each frame ends with a 33 ms sleep targeting

30 FPS, ensuring consistent gameplay regardless of CPU speed.

Entities (bullets and enemies) are managed via static arrays with `active` flags. Spawning finds a free slot; destruction clears the flag without memory movement. This mirrors how an OS reuses process table entries.

C. I/O Management

Input. The terminal is switched from cooked mode (line-buffered, echoed) to raw mode using the POSIX `termios` interface. In raw mode, `getchar()` returns immediately and silently, enabling non-blocking key capture. Arrow keys, which send 3-byte ANSI escape sequences, are decoded byte-by-byte. A bitmask drain function reads all pending keystrokes per frame and ORs them into a single state word, enabling simultaneous key detection for diagonal movement.

Output. The renderer maintains two character buffers: a back buffer (being drawn) and a front buffer (currently displayed). Each frame, the game clears the back buffer, draws all objects into it in Z-order (border, enemies, bullets, player, HUD), then calls a flip function. The flip compares every cell of back versus front and emits ANSI cursor-positioning commands only for changed cells. This differential approach typically sends fewer than 50 bytes to the terminal per frame, completely eliminating flicker. The engine also uses the terminal's alternate screen buffer to preserve the user's original terminal content.

D. File System

The game persists three types of data: registered usernames (`users.txt`), the current session identity (`current_user.txt`), and a sorted top-10 leaderboard (`leaderboard.txt`). All file operations use POSIX system calls—`open()`, `read()`, `write()`, and `close()` from `<fcntl.h>` and `<unistd.h>`—rather than `<stdio.h>` wrappers.

Files are read into character buffers and parsed manually, splitting on spaces and newlines using custom `str_to_int()` and `my_strcmp()` functions. When a new score is saved, the leaderboard is re-sorted via insertion sort and truncated to the top 10 entries before being written back. On boot, the engine reads `current_user.txt` to restore the previous session, allowing returning players to skip the registration prompt.

E. Error Handling and Security

The engine addresses several failure modes. Memory exhaustion in `my_alloc()` triggers a diagnostic message and immediate `exit(1)`, preventing undefined behavior. The rendering function validates all coordinates via a bounds-checking utility before writing to the screen buffer, silently discarding out-of-range draws. The username prompt sanitizes input by auto-lowercasing letters and rejecting all characters except `a--z` and `0--9`, preventing malformed data from entering persistent files.

Terminal state corruption is prevented by two safeguards: `atexit()` registers a restore function during keyboard initialization, ensuring raw mode is reverted even on abnormal exit, and `screen_restore()` is called at the end of `main()` to return cursor visibility and the primary screen buffer. File I/O operations check all return values; missing files are treated as empty data rather than fatal errors, and the leaderboard parser skips malformed lines without invalidating the rest of the file.

F. Networking Model

Raiden is a standalone, single-machine application and does not currently implement networking. All state is local and all persistence uses the local filesystem. A future version could introduce BSD sockets for multiplayer, using UDP for real-time game state and TCP for leaderboard synchronization.

G. User Interface

The UI is state-driven: each `GameState` renders a full-screen ASCII layout within the 120×38 grid. Screens include an animated title banner, a validated username prompt, a navigable menu with W/S and Enter, the real-time gameplay view with a HUD (score and health bar), a game over screen showing rank and replay options, and a top-10 leaderboard highlighting the current player. Visual feedback includes a rapid flash effect when the player is hit, a glyph change on damaged enemies, and floating damage numbers above struck enemies.

III. FUTURE SCOPE

Several extensions are planned to expand the engine's capabilities:

- 1) **Threaded Rendering:** Decoupling game logic from the rendering pipeline using POSIX threads (`pthread`s) would allow the logic to update at a higher rate while rendering runs independently, improving responsiveness on slower terminals.
- 2) **Binary Persistence:** Replacing the text-based file format with a custom binary serialization would reduce file sizes and eliminate the overhead of character-by-character parsing.
- 3) **Sound Output:** Basic audio feedback could be achieved through the terminal bell character (`\a`) for hit events, or by writing PCM data to `/dev/audio` via raw file descriptors.
- 4) **Networked Multiplayer:** BSD sockets could enable real-time co-op gameplay, using UDP for low-latency game state synchronization and TCP for reliable leaderboard updates.
- 5) **Allocator Enhancements:** Adding block coalescing (merging adjacent free blocks) would support variable-size allocations, enabling more complex entity types and dynamic level data.

IV. CONCLUSION

The Raiden engine demonstrates that core OS principles—pool-based memory management, cooperative scheduling, raw device I/O, and POSIX file operations—can be applied to build a complete real-time application from first principles. The custom memory pool eliminates fragmentation and provides deterministic allocation times. The double-buffered renderer achieves flicker-free output by transmitting only differential updates. Raw terminal mode enables responsive, non-blocking input with simultaneous key detection. POSIX file descriptors provide direct, controlled access to persistent storage without the overhead of buffered I/O wrappers. Together, these subsystems produce a playable, robust arcade game in approximately 2,000 lines of C, validating the constraint-driven design methodology.

REFERENCES

- [1] IEEE Std 1003.1-2017, *IEEE Standard for Information Technology—Portable Operating System Interface (POSIX)*, IEEE, 2017.
- [2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013.
- [3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015.
- [4] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.